



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

FACULTAD DE CIENCIAS TÉCNICAS

CARRERA DE INGENIERÍA EN CIBERSEGURIDAD

AUTÓNOMO 2

PROYECTO INTEGRADOR

TEMA DE EVALUACIÓN: Sistema de Gestión de Libros bajo el Paradigma de Programación Orientada a Objetos en Golang: Etapa 2. Desarrollo del Software

Profesor: Héctor Guillermo Avalos Silva

Asignatura: Programación Orientada a Objetos Autores:

Valenzuela Saavedra Andrés Fernando Fecha: 13 de junio del
2026



1	Introducción y Objetivos del Sistema	2
1.1	Objetivo General.....	2
1.2	Objetivos Específicos	2
2	Alcance del Proyecto	2
2.1	Límites Funcionales (Alcance Dentro del Proyecto).....	2
2.2	Límites No Funcionales y Exclusiones (Fuera del Alcance)	3
3	Estructura de Funcionalidades por Módulo	3
3.1	Descripción Operativa de los Módulos	4
4	Arquitectura de Software y Programación Funcional en Go	4
4.1	Conceptos Clave Aplicados.....	4
4.2	Implementación de Funciones de Orden Superior Genéricas.....	4
4.3	Composición Criptográfica Funcional	5
5	Paquetes y Dependencias (Ecosistema de Software)	6
5.1	Paquetes de la Biblioteca Estándar de Go (Sin dependencias externas).....	6
5.2	Paquetes de Terceros Documentados	6
6	Configuración del Repositorio de GitHub	6
6.1	Estructura del Proyecto en el Repositorio	7
6.2	Pipeline de Integración Continua (GitHub Actions).....	7
7	Características de Entrega y Planificación del Video	8
7.1	Estructura Planificada del Video Explicativo	8

Índice

1	Introducción y Continuación del Plan
1.1	Contexto del Autónomo 1 y Evolución Arquitectónica
1.2	Objetivo General
1.3	Objetivos Específicos
2	Paso 1: Análisis Teórico-Práctico
2.1	Evidencia e Interpretación de Fuentes
2.2	Posición del Estudiante
2.3	Conclusiones y Logros Relacionados
3	Paso 2: Desarrollo del Sistema
3.1	Arquitectura Hexagonal
3.2	Principios SOLID Aplicados
3.3	Entidades del Dominio (una por archivo)
3.4	Encapsulación, Interfaces y Polimorfismo
3.5	Manejo de Errores
3.6	Interfaz de Consola con Menú Interactivo
3.7	Funcionalidades Implementadas y Pendientes
4	Estructura del Repositorio
5	Planificación de Videos de Entrega
6	Referencias



1 Introducción y Continuación del Plan

En el Autónimo 1 se planificó el Sistema de Gestión de Libros Electrónicos bajo programación funcional en Golang. La presente entrega (Etapa 2) continúa ese plan aplicando Programación Orientada a Objetos, arquitectura hexagonal, principios SOLID, estructuras de datos, interfaces y un adaptador de entrada con menú interactivo en consola.

El sistema no está finalizado; este documento reporta un avance significativo con catálogo bibliográfico operativo, gestión de usuarios, ciclo de préstamos/devoluciones y una CLI funcional. Quedan pendientes persistencia en BD, autenticación JWT y módulos criptográficos del Autónimo 1.

1.1 Contexto del Autónimo 1 y Evolución Arquitectónica

La Etapa 1 definió módulos de catálogo, seguridad, acceso y auditoría bajo pipelines funcionales. En la Etapa 2 esos módulos se reorganizan bajo arquitectura hexagonal: el dominio queda aislado en el centro, los puertos definen contratos y los adaptadores conectan consola (entrada) y memoria RAM (salida).

Además, se aplicaron los principios SOLID: cada entidad en su propio archivo (SRP), interfaces segregadas por contexto — catálogo, usuario, préstamo — (ISP), e inversión de dependencias mediante puertos outbound (DIP). Esto permite agregar API REST o PostgreSQL sin reescribir el núcleo (OCP).

1.2 Objetivo General

Desarrollar un avance funcional del Sistema de Gestión de Libros en Golang con arquitectura hexagonal y SOLID, encapsulación, interfaces, manejo de errores y menú de consola interactivo, alineado al plan del Autónimo 1.

1.3 Objetivos Específicos

- Organizar el código en capas hexagonal: domain, port, application, adapter.
- Aplicar SOLID: un archivo por entidad/servicio, interfaces segregadas, inversión de dependencias.
- Implementar adaptador inbound CLI con menú de 13 opciones operativas.
- Modelar Libro, Autor, Usuario, Préstamo y Revista con encapsulación en domain/entity/.
- Definir puertos inbound/outbound e implementar persistencia en memoria como adaptador outbound.
- Documentar arquitectura en docs/ARQUITECTURA.md y guía de uso en README.md.

2 Paso 1: Análisis Teórico-Práctico

2.1 Evidencia e Interpretación de Fuentes

Cockburn (2005) propone la arquitectura hexagonal para aislar la lógica de negocio mediante puertos y adaptadores. Aplicada al proyecto, CatalogoServicio depende del puerto outbound.Catalogo, no de la implementación Memoria — cumpliendo DIP.

Martin (2000) define SOLID como base del diseño orientado a objetos mantenible. SRP justifica separar catalogo_servicio.go de prestamo_servicio.go; ISP justifica dividir Biblioteca en CatalogoCasosUso, UsuarioCasosUso y PrestamoCasosUso en lugar de una interfaz monolítica.

Donovan y Kernighan (2015) y Bodner (2021) confirman que Go implementa POO mediante structs, métodos, visibilidad por paquete e interfaces implícitas — base técnica de encapsulación y polimorfismo.

2.2 Posición del Estudiante

Tesis: La combinación de arquitectura hexagonal + SOLID en Go produce un sistema bibliotecario



extensible sin herencia clásica. Cada capa tiene responsabilidad única; la CLI es un adaptador reemplazable; el dominio permanece puro.

Límite reconocido: la encapsulación en Go depende del paquete, no de modificadores `private` por campo. El menú CLI es síncrono y sin persistencia — adecuado para el avance académico actual.

2.3 Conclusiones y Logros Relacionados

El análisis sustenta que hexagonal + SOLID facilita evolucionar hacia API REST y PostgreSQL sin modificar `domain/entity` ni `application/*_servicio`. Logro inmediato: sistema usable vía menú con registro, búsqueda, préstamo, devolución y datos demo.

Prioridades futuras: (1) adaptador outbound PostgreSQL, (2) adaptador inbound HTTP/REST, (3) autenticación JWT, (4) cifrado e integridad de archivos digitales.

3 Paso 2: Desarrollo del Sistema

3.1 Arquitectura Hexagonal

El sistema sigue el patrón Ports & Adapters. Capas: (1) `domain` — entidades y errores; (2) `port` — contratos inbound (driving) y outbound (driven); (3) `application` — casos de uso; (4) `adapter` — CLI (entrada) y Memoria (salida). Las dependencias apuntan siempre hacia el dominio.

```
CLI (adapter/inbound/cli)
  ↓ port/inbound
GestorBiblioteca + *_servicio (application)
  ↓ port/outbound
Memoria (adapter/outbound/persistence)
  ↓
domain/entity (Libro, Usuario, Prestamo...)
```

3.2 Principios SOLID Aplicados

Principio	Implementación en el proyecto
SRP	Un archivo por entidad, servicio, handler y repositorio parcial
OCP	Nuevos adaptadores (HTTP, PostgreSQL) sin modificar <code>application/</code>
LSP	Libro y Revista sustituyen a Recurso en préstamos y listados
ISP	Puertos inbound/outbound segregados por catálogo, usuario y préstamo
DIP	Servicios dependen de <code>port/outbound</code> , no de <code>persistence.Memoria</code>

3.3 Entidades del Dominio (una por archivo)

Cuadro 2: Entidades en `internal/domain/entity/` — una responsabilidad por archivo (SRP):

Archivo	Entidad / Interfaz	Responsabilidad
<code>recurso.go</code>	Recurso (interface)	Contrato polimórfico de catálogo
<code>autor.go</code>	Autor	Creador de una obra
<code>libro.go</code>	Libro	Obra impresa con ISBN
<code>revista.go</code>	Revista	Publicación periódica
<code>usuario.go</code>	Usuario	Lector registrado
<code>prestamo.go</code>	Prestamo	Ciclo préstamo-devolución

3.4 Encapsulación, Interfaces y Polimorfismo

Campos privados (minúscula) en `domain/entity/` con getters controlados. La interfaz Recurso permite que Libro y Revista participen en `ListarRecursosDisponibles` y `PrestarRecurso` sin conocer el tipo



concreto — polimorfismo.

```
// domain/entity/recurso.go
type Recurso interface {
    ID() string; Titulo() string; Disponible() bool
    MarcarPrestado(); MarcarDevuelto(); Descripcion() string
}
```

3.5 Manejo de Errores

Errores tipados segregados en domain/errores/ (catalogo.go, usuario.go, prestamo.go, validacion.go). Propagación con `fmt.Errorf("contexto: %w", err)` en `application/*_servicio.go`.

3.6 Interfaz de Consola con Menú Interactivo

El adaptador inbound cli/ expone un menú interactivo con `bufio.Reader`. Handlers segregados por dominio: `handlers_catalogo.go`, `handlers_usuario.go`, `handlers_prestamo.go` (SRP). Ejecución: `go run ./cmd/main.go`

Opciones del menú:

- 1–2: Registrar libro / revista
- 3: Registrar usuario
- 4–5: Buscar por título / autor
- 6–8: Listar libros, recursos disponibles, usuarios
- 9–10: Prestar / devolver recurso
- 11–12: Préstamos activos / resumen del catálogo
- 13: Cargar datos de demostración
- 0: Salir

3.7 Funcionalidades Implementadas y Pendientes

Implementadas:

- Arquitectura hexagonal con 4 capas y 32 archivos Go.
- Servicios segregados: `CatalogoServicio`, `UsuarioServicio`, `PrestamoServicio`, `ConsultaServicio`.
- Menú CLI interactivo con 13 opciones.
- Registro, búsqueda, préstamo, devolución y resumen de catálogo.
- Documentación: `README.md` y `docs/ARQUITECTURA.md`.

Pendientes:

- Adaptador outbound PostgreSQL.
- Adaptador inbound API REST.
- Autenticación JWT y cifrado AES-GCM (Autónomo 1).
- Ledger de auditoría y CI/CD.

4 Estructura del Repositorio

```
sistema-gestion-libros/
├── cmd/main.go
├── docs/
│   ├── ARQUITECTURA.md
│   ├── Autonomo1POOAFVS.pdf
│   └── Autonomo2POOAFVS.pdf
├── README.md
├── internal/
│   ├── domain/entity/      # autor.go, libro.go, revista.go...
│   ├── domain/errores/
│   ├── port/inbound/      # catalogo, usuario, prestamo
│   ├── port/outbound/
│   └── application/       # *_servicio.go, gestor_biblioteca.go
```



```
└─ adapter/  
  └─ inbound/cli/    # menu.go, handlers_*.go  
    └─ outbound/persistence/ # memoria_*.go
```

5 Referencias

Bodner, J. (2021). Learning Go. O'Reilly Media.

Cockburn, A. (2005). Hexagonal architecture. Alistair Cockburn.

Donovan, A. A., y Kernighan, B. W. (2015). The Go programming language. Addison-Wesley.

Martin, R. C. (2000). Design principles and design patterns. Object Mentor.

Pike, R. (2012). Go at Google I/O 2012. Google I/O.

OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/www-project-top-ten/>